Project on Hockey Analysis

By: Nikhitha Chenna

Table of Contents

1	Introduction		3
	1.1	Setup Checklist for Project	3
	1.2	Architecture	3
2	Problem Statement		4
	2.1	Objective	4
	2.2	Abstract of the project	4
	2.3	Technology use	4
3	Implementation in PYSPARK		5
	3.1	Summary of the functionality to be built	5
	3.2	Guidelines on the functionality to be built	5
	3.3	Implementation of project	6
4	Reports to be built		26
5	Appendix (source Files)		26

1 INTRODUCTION

This project demonstrates the implementation of an end-to-end Azure Data Engineering pipeline using Azure services and Databricks. The project focuses on processing hockey datasets, transforming raw data into fact and dimension tables, and generating analytical reports using PySpark and Spark SQL.

1.1 Setup Checklist for Project

- Azure Storage Account Created
- Azure Data Lake Storage Gen2 Enabled
- Azure Databricks Workspace Created
- Azure Data Factory Created
- Blob Container Configured
- CSV Files Uploaded
- Databricks Cluster Started
- ADLS Mounted in Databricks
- PySpark Environment Configured

1.2 Architecture of project

CSV Files
↓
Azure Blob Storage
↓
Azure Data Factory
↓
Azure Data Lake Storage Gen2
↓
Azure Databricks
↓
Dimension & Fact Tables
↓
Spark SQL Reports
↓
Output Folder in ADLS

2 Problem Statement

2.1 Objective

To process hockey datasets stored in Azure Data Lake Storage using Azure Databricks and generate meaningful analytical reports using PySpark and Spark SQL.

2.2 Abstract of the Project

This project uses Azure Data Lake Storage Gen2, Azure Databricks, and PySpark to process football datasets. Raw CSV files are ingested into Databricks, transformed into dimension and fact tables, and used to generate analytical reports such as top goal scorers, richest clubs, and players with maximum appearances. The final reports are stored in the output folder of ADLS**.**

2.3 Technologies Used

- Azure Blob Storage
- Azure Data Lake Storage Gen2
- Azure Databricks
- Azure Data Factory
- PySpark
- Spark SQL
- Delta Tables

3 IMPLEMENTATION IN PYSPARK

3.1 Summary of Functionalities Built

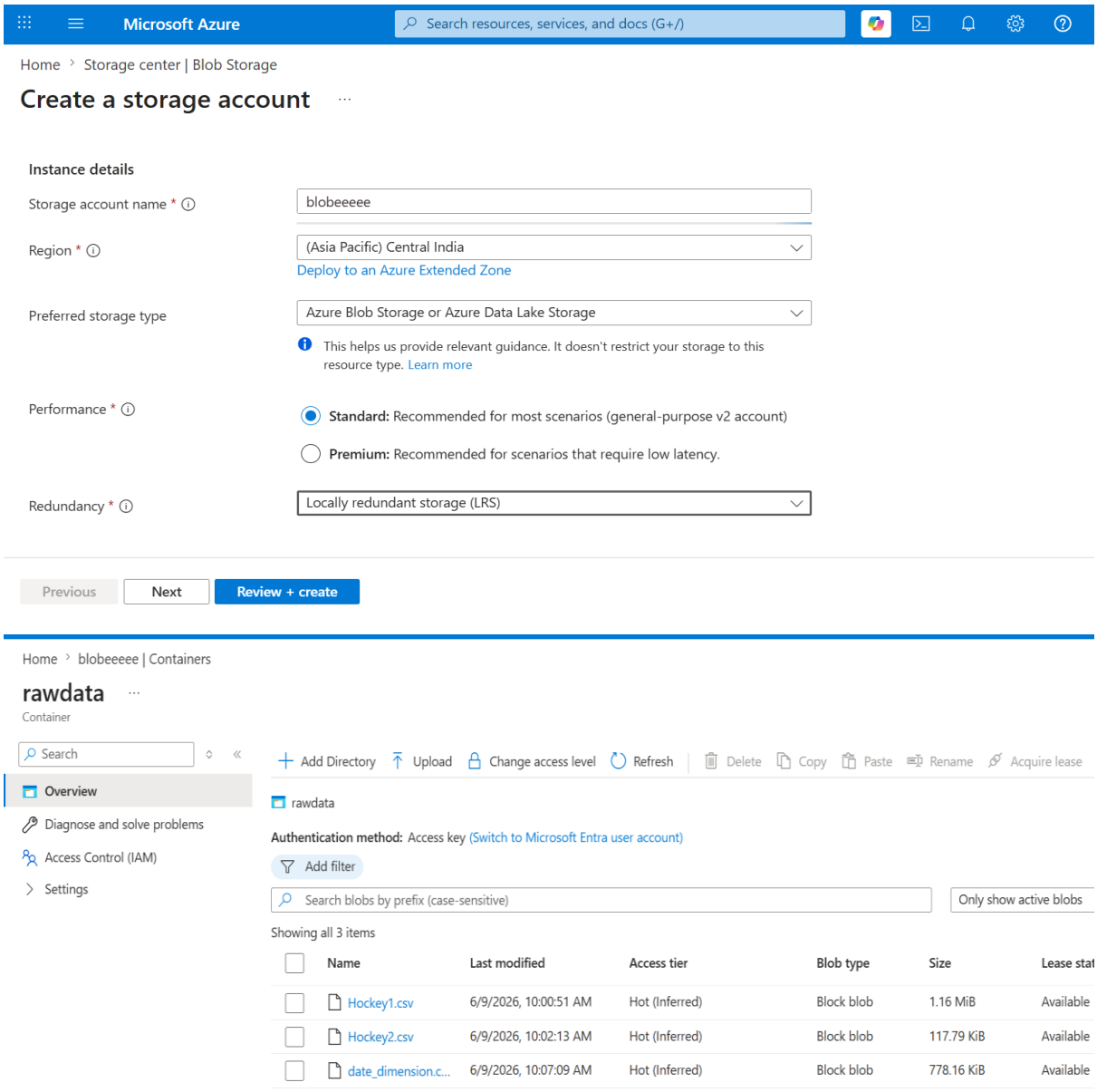
- Mounted Azure Data Lake Storage in Databricks
- Read CSV files using PySpark
- Created Dimension Tables
- Created Fact Tables
- Performed ETL transformations
- Generated reports using Spark SQL
- Saved reports to ADLS output folder

3.2 Guidelines on Functionalities Built

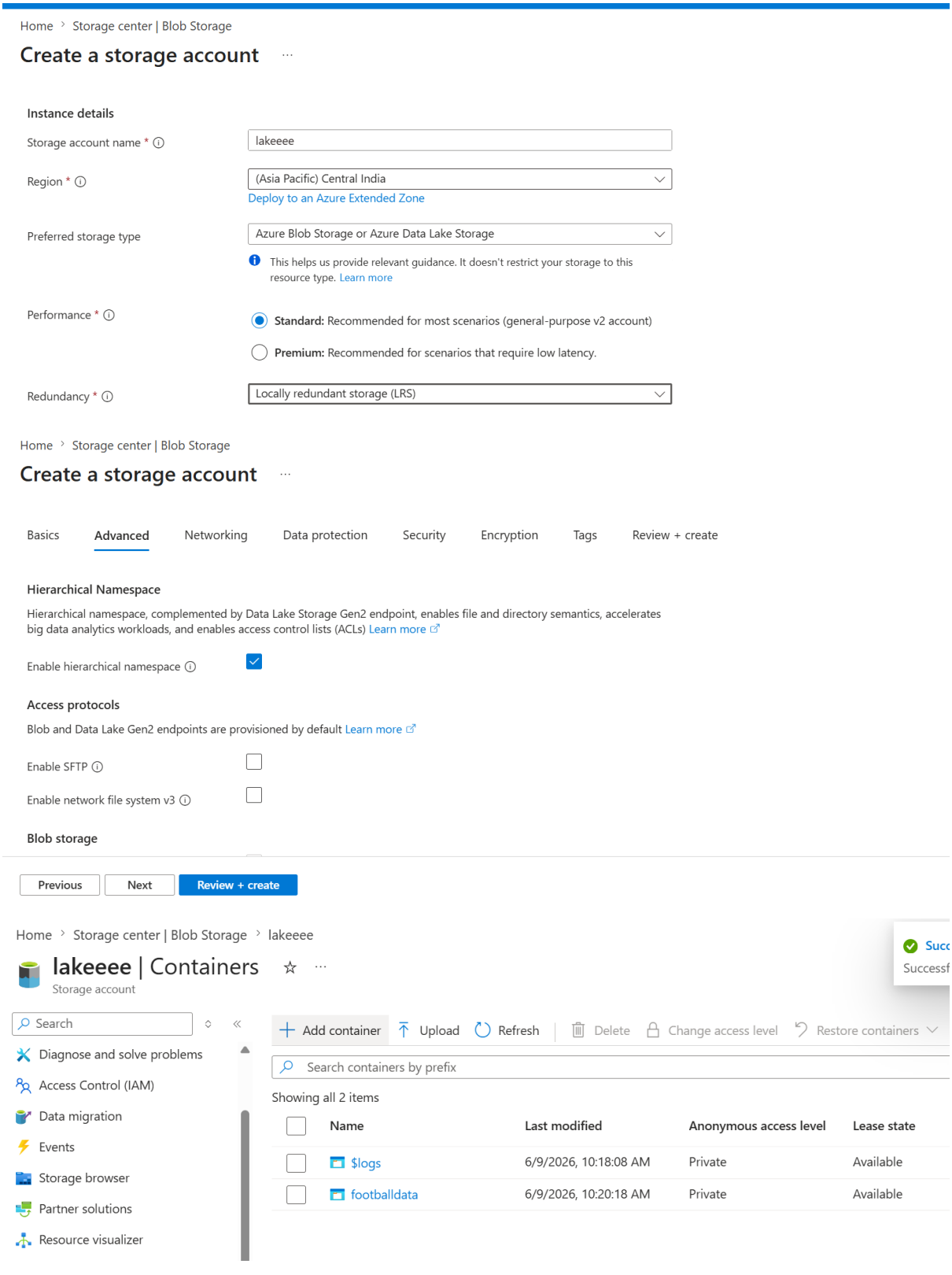
- Step 1: Read Source Files
  Loaded Hockey1.csv, Hockey2.csv, and date_dimension.csv into Databricks.
- Step 2: Create Dimension Tables
  Created dim_player, dim_club, dim_country, dim_match, and dim_date tables.
- Step 3: Create Fact Tables
  Created fact_player_stats and fact_team_stats tables using transformed datasets.
- Step 4: Create Reports
  Generated analytical reports using Spark SQL queries.
- Step 5: Save Reports
  Stored final report outputs into Azure Data Lake Storage output folder.

3.3 Implementation of project

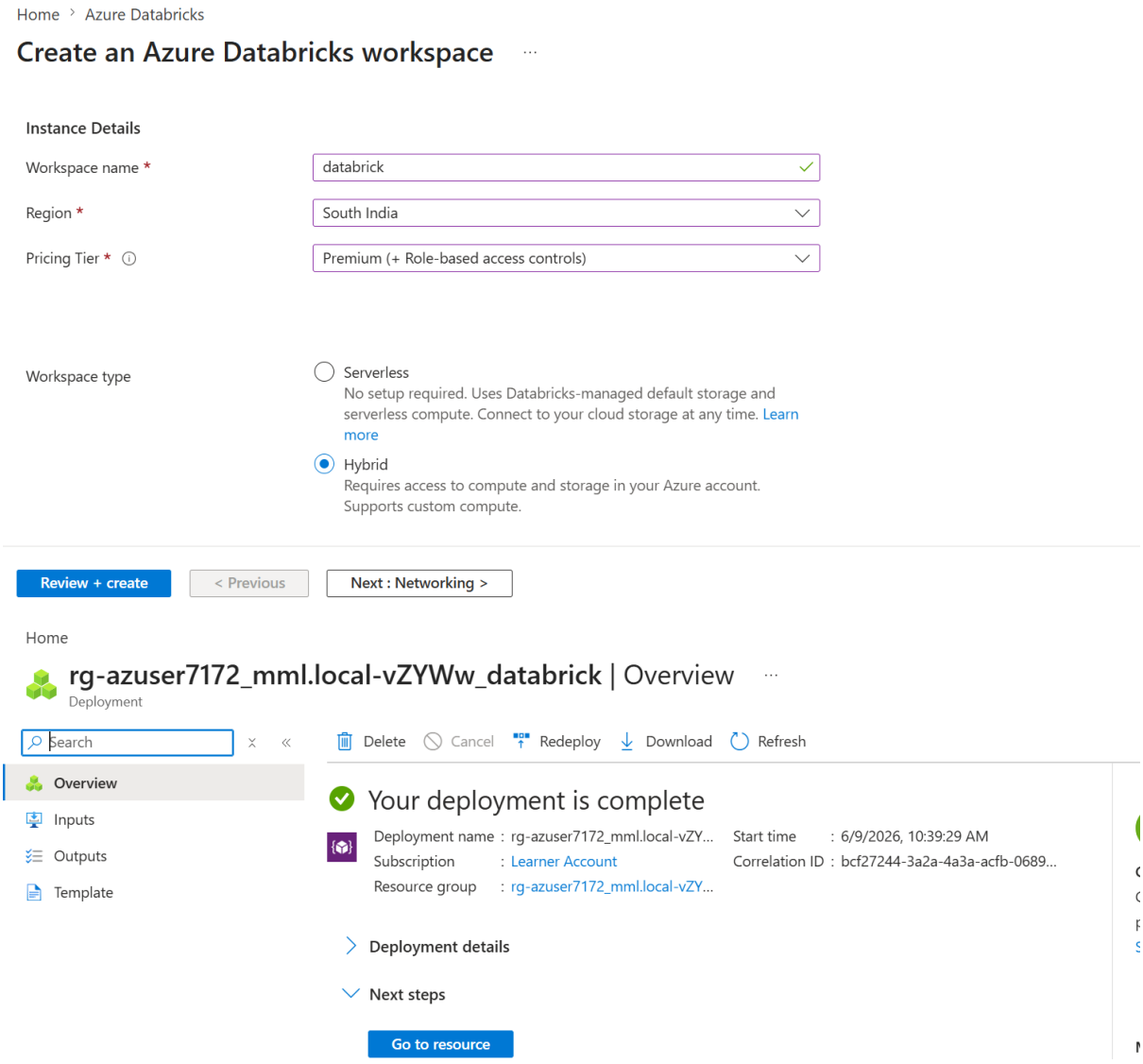
Azure Blob Storage container was created successfully and the hockey dataset CSV files were uploaded into the rawdata container for further processing.

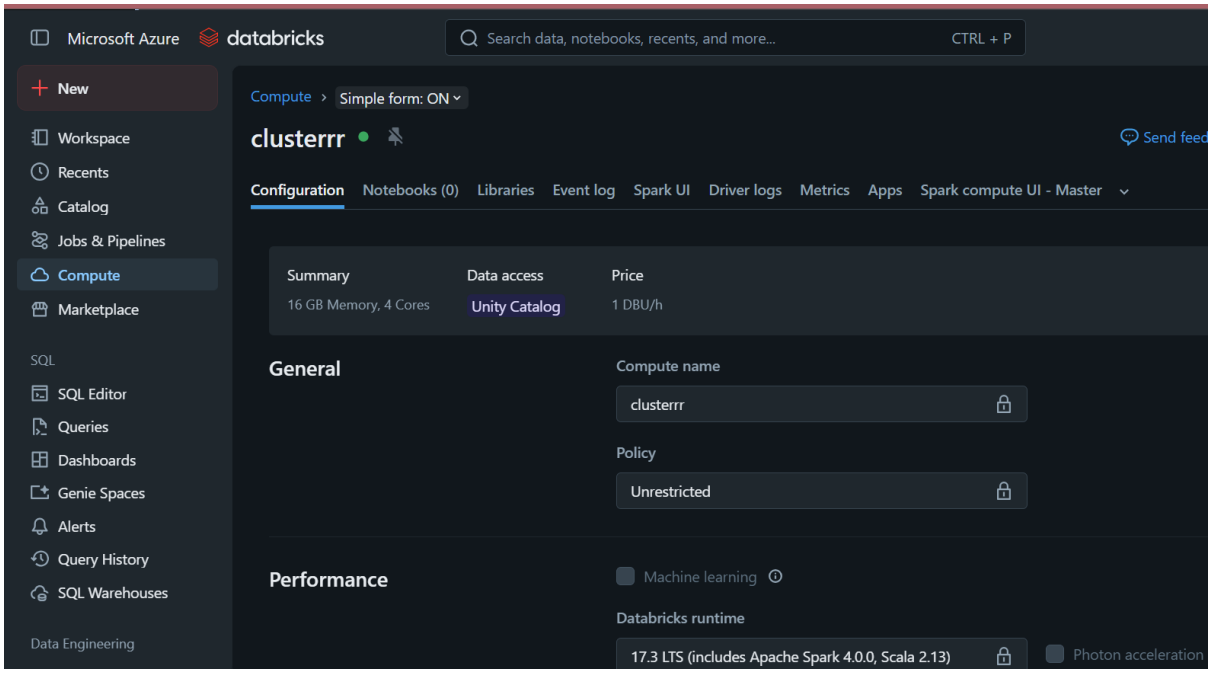


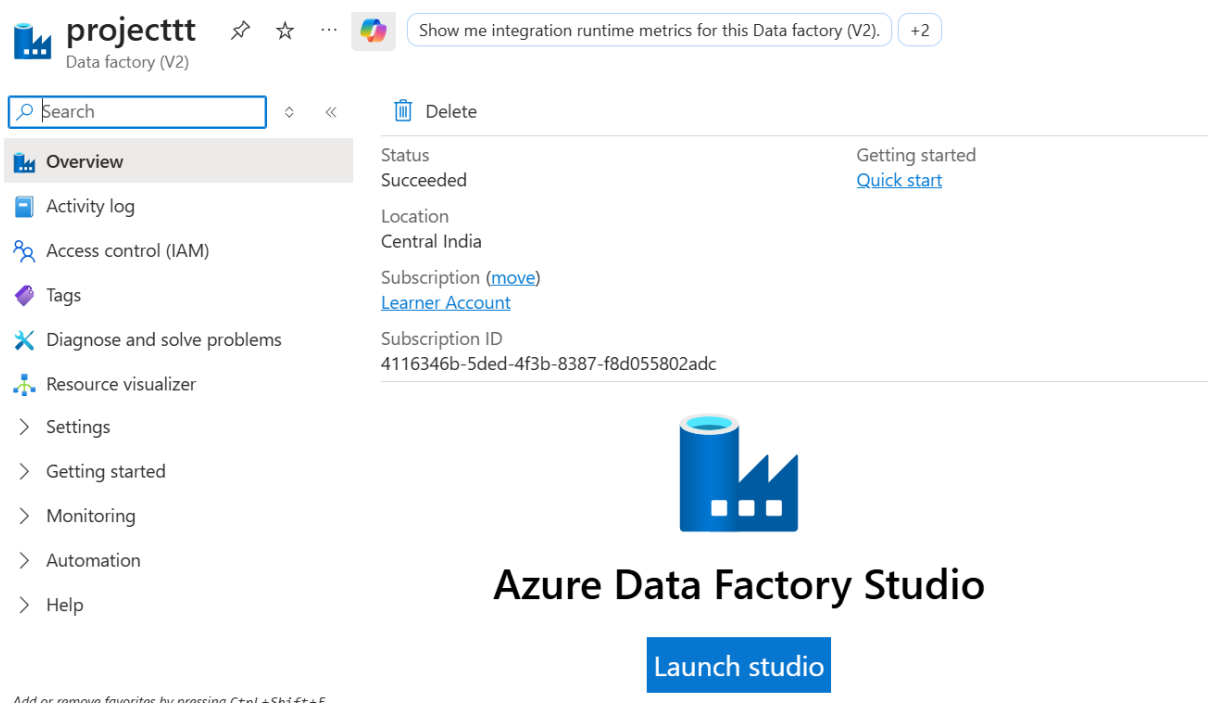
Azure Data Lake Storage Gen2 was configured to store raw, processed, and transformed data generated during the ETL process.

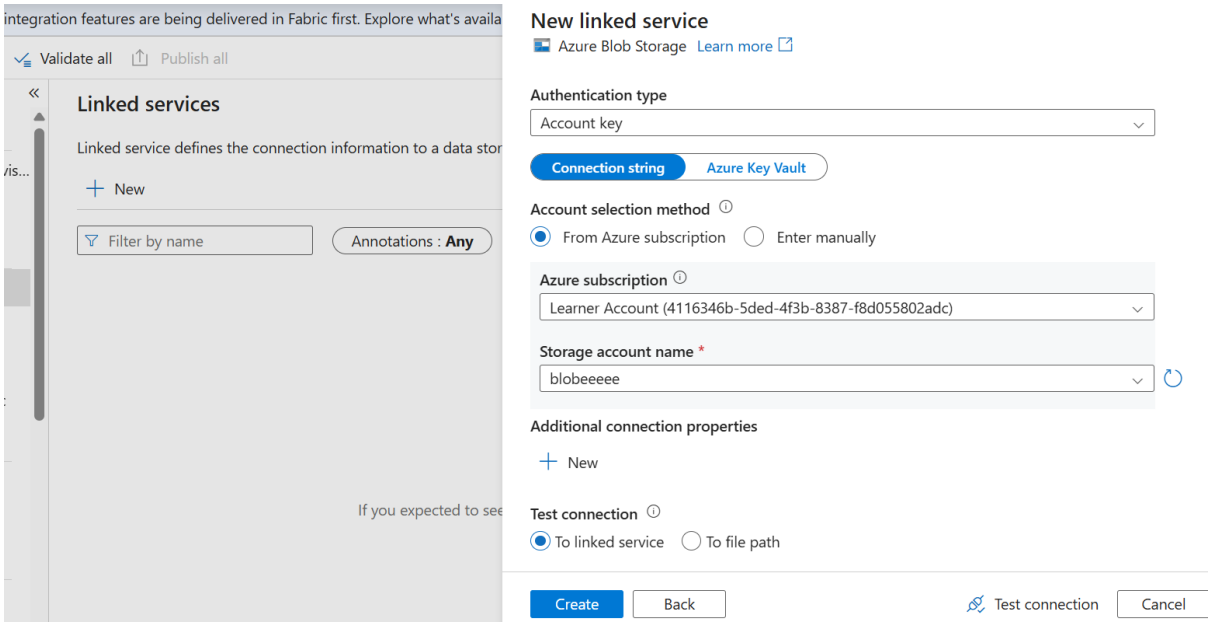


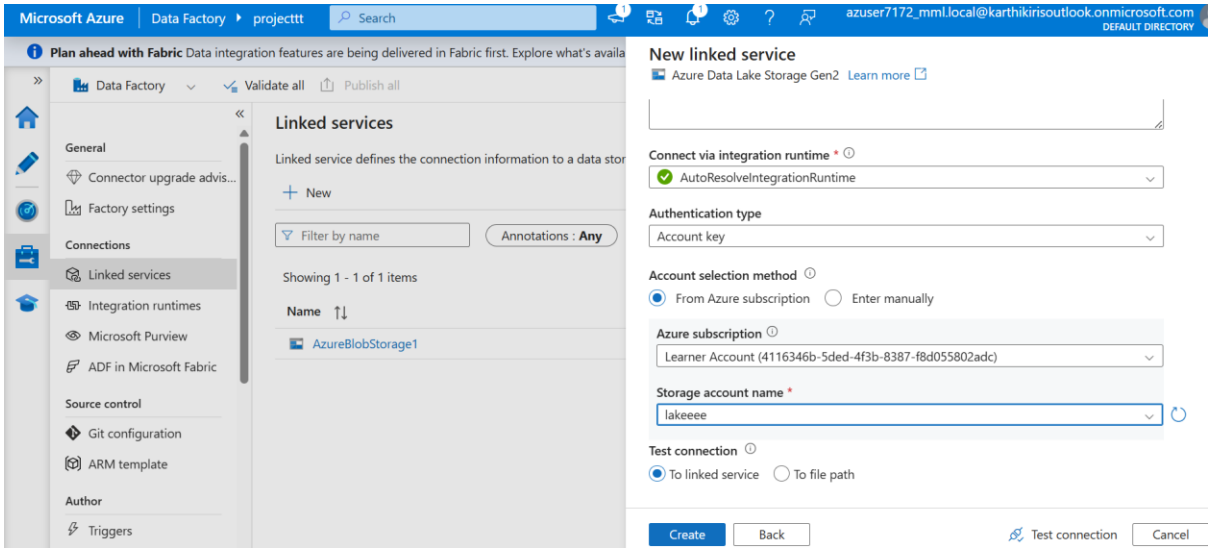
Azure Databricks cluster was created and configured successfully to perform distributed Spark-based data transformations.




Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie Spaces
Alerts
Query History
SQL Warehouses
Data Engineering
Compute › Simple form: ON
clusterrr
Send feed
Configuration
Notebooks (0)
Libraries
Event log
Spark UI
Driver logs
Metrics
Apps
Spark compute UI - Master
Summary
16 GB Memory, 4 Cores
Data access
Unity Catalog
Price
1 DBU/h
General
Compute name
clusterrr
Policy
Unrestricted
Performance
Machine learning
Databricks runtime
17.3 LTS (includes Apache Spark 4.0.0, Scala 2.13)
Photon acceleration


projecttt
Data factory (V2)
Show me integration runtime metrics for this Data factory (V2).
+2
Search
Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Resource visualizer
Settings
Getting started
Monitoring
Automation
Help
Delete
Status
Succeeded
Getting started
Quick start
Location
Central India
Subscription (move)
Learner Account
Subscription ID
4116346b-5ded-4f3b-8387-f8d055802adc
Azure Data Factory Studio
Launch studio





Linked services

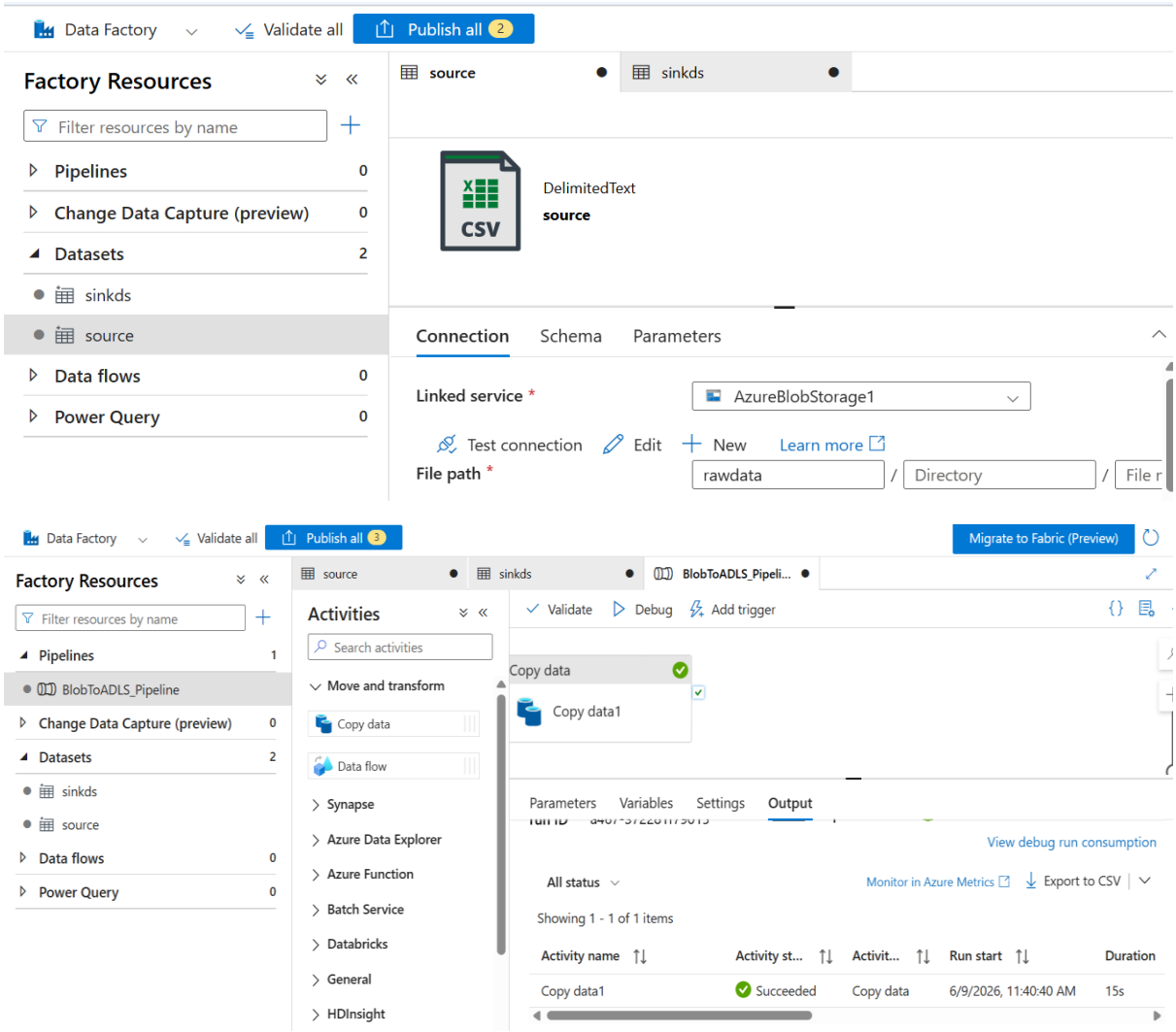
Linked service defines the connection information to a data store or compute. Learn more

New

Filter by name | Annotations : Any

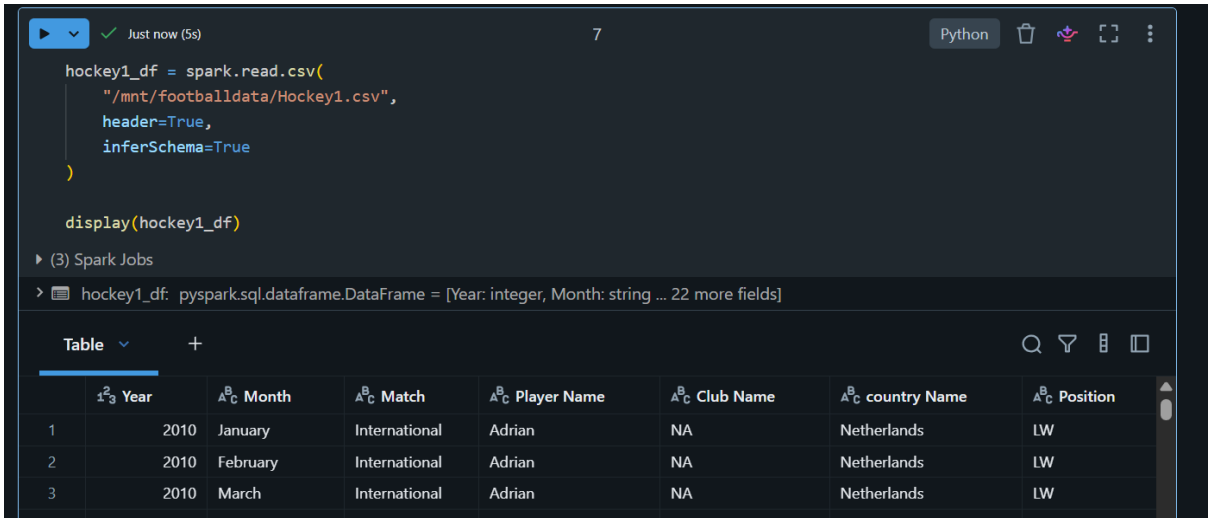
Showing 1 - 2 of 2 items

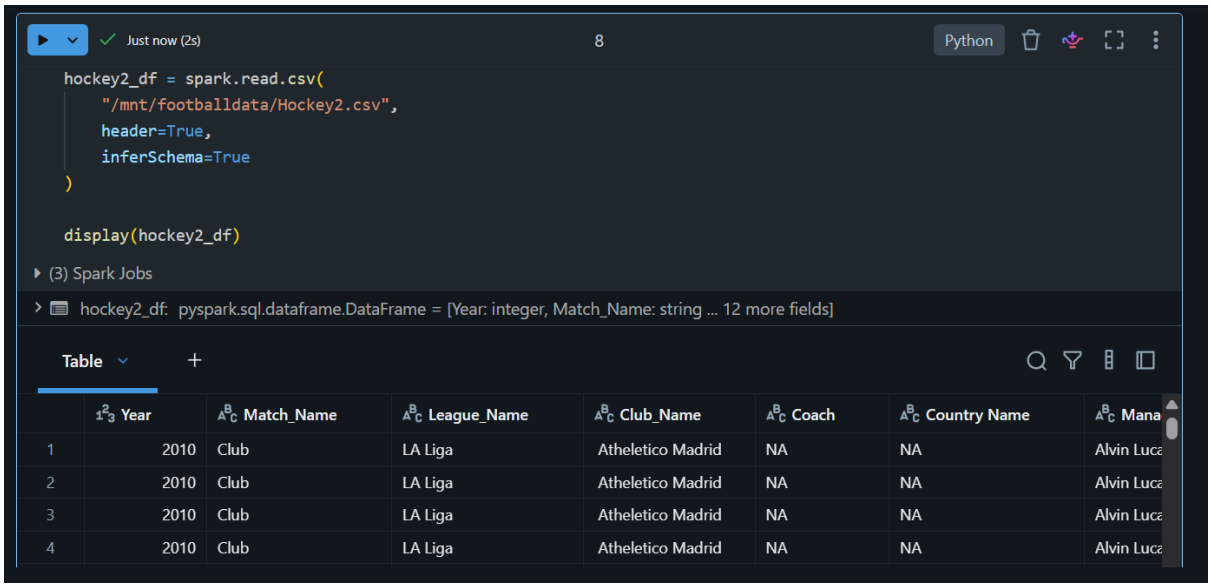
Name	Type	Related	Annotations
AzureBlobStorage1	Azure Blob Storage	1	
AzureDataLakeStorage1	Azure Data Lake Storage Gen2	1	



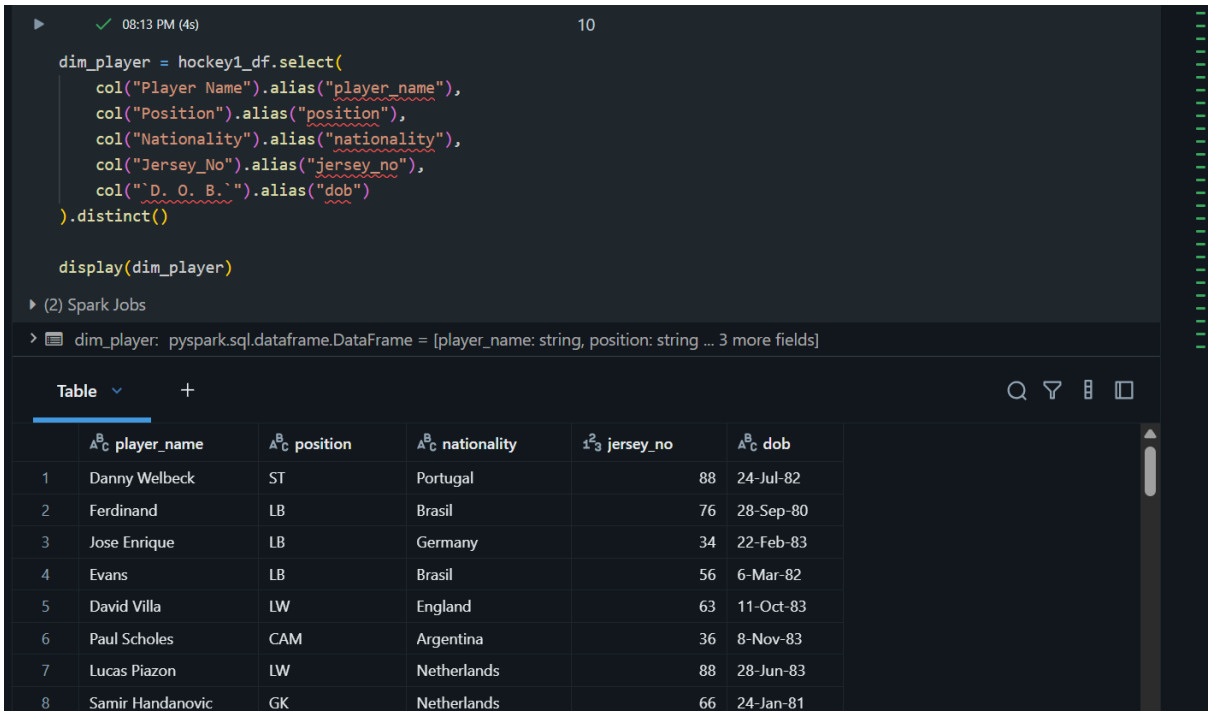
Azure Data Lake Storage was mounted successfully into Databricks using DBFS mount points for secure and efficient data access.

The hockey datasets were loaded into Spark DataFrames using PySpark for transformation and analytical processing.





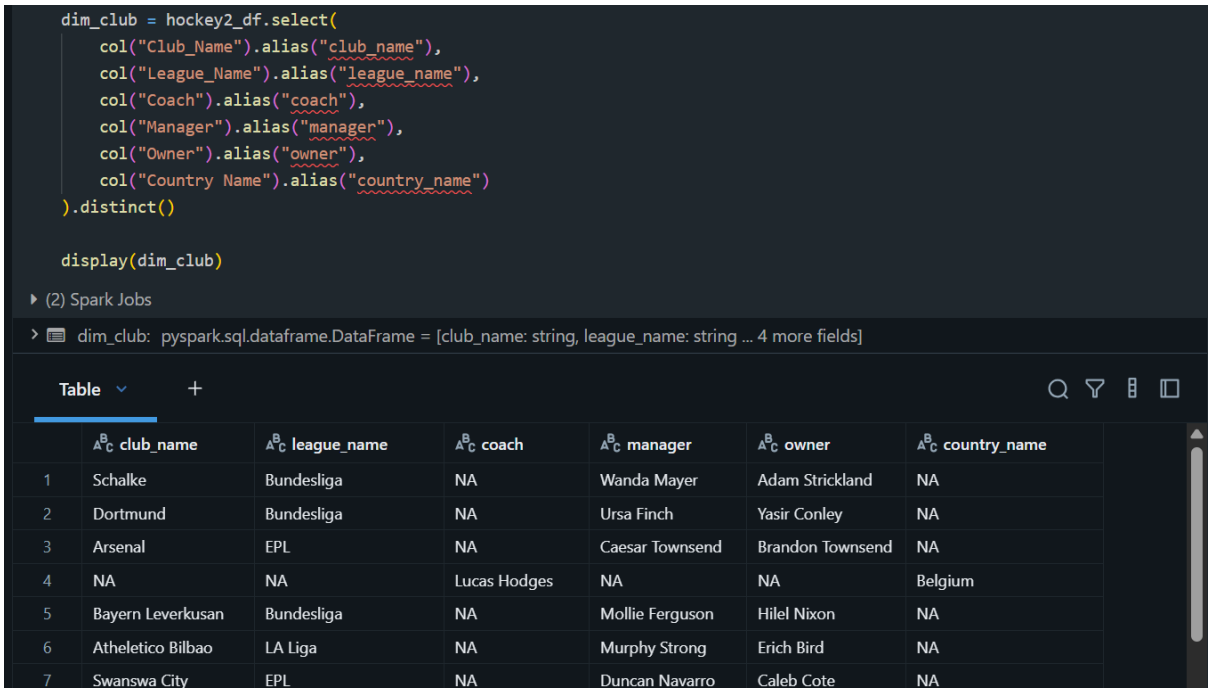
Dimension tables were created to store descriptive information related to players, clubs, countries, matches, and dates. These tables help in organizing and analysing hockey data efficiently.



```
08:13 PM (4s)                                                    10

dim_player = hockey1_df.select(
    col("Player Name").alias("player_name"),
    col("Position").alias("position"),
    col("Nationality").alias("nationality"),
    col("Jersey_No").alias("jersey_no"),
    col("`D. O. B.`").alias("dob")
).distinct()

display(dim_player)

▸ (2) Spark Jobs
▸ dim_player:  pyspark.sql.dataframe.DataFrame = [player_name: string, position: string ... 3 more fields]
```

Table +

	player_name	position	nationality	jersey_no	dob
1	Danny Welbeck	ST	Portugal	88	24-Jul-82
2	Ferdinand	LB	Brasil	76	28-Sep-80
3	Jose Enrique	LB	Germany	34	22-Feb-83
4	Evans	LB	Brasil	56	6-Mar-82
5	David Villa	LW	England	63	11-Oct-83
6	Paul Scholes	CAM	Argentina	36	8-Nov-83
7	Lucas Piazon	LW	Netherlands	88	28-Jun-83
8	Samir Handanovic	GK	Netherlands	66	24-Jan-81



```
dim_club = hockey2_df.select(
    col("Club_Name").alias("club_name"),
    col("League_Name").alias("league_name"),
    col("Coach").alias("coach"),
    col("Manager").alias("manager"),
    col("Owner").alias("owner"),
    col("Country Name").alias("country_name")
).distinct()

display(dim_club)

▸ (2) Spark Jobs
▸ dim_club:  pyspark.sql.dataframe.DataFrame = [club_name: string, league_name: string ... 4 more fields]
```

Table +

	club_name	league_name	coach	manager	owner	country_name
1	Schalke	Bundesliga	NA	Wanda Mayer	Adam Strickland	NA
2	Dortmund	Bundesliga	NA	Ursa Finch	Yasir Conley	NA
3	Arsenal	EPL	NA	Caesar Townsend	Brandon Townsend	NA
4	NA	NA	Lucas Hodges	NA	NA	Belgium
5	Bayern Leverkusan	Bundesliga	NA	Mollie Ferguson	Hilel Nixon	NA
6	Atheletico Bilbao	LA Liga	NA	Murphy Strong	Erich Bird	NA
7	Swanswa City	EPL	NA	Duncan Navarro	Caleb Cote	NA

```
dim_country = hockey1_df.select(
    col("country Name").alias("country_name")
).distinct()

display(dim_country)
```

(2) Spark Jobs

dim_country: pyspark.sql.dataframe.DataFrame = [country_name: string]

Table +

	country_name
1	Germany
2	NA
3	Argentina
4	Belgium
5	Italy
6	Spain
7	Mexico
8	Brasil
9	England

```
dim_match = hockey1_df.select(
    col("Match").alias("match_type")
).distinct()

display(dim_match)
```

(2) Spark Jobs

dim_match: pyspark.sql.dataframe.DataFrame = [match_type: string]

Table +

	match_type
1	International
2	Club

```
dim_date = dates_df.select(
    "date_key",
    "full_date",
    "day_name",
    "month_name",
    "fb_year",
    "quarter"
)

display(dim_date)
```

(1) Spark Jobs

dim_date: pyspark.sql.dataframe.DataFrame = [date_key: integer, full_date: timestamp ... 4 more fields]

Table +

	date_key	full_date	day_name	month_name	fb_year	quarter
1	20080101	2008-01-01T00:00:00.000+00:00	Tuesday	January	2008	1
2	20080102	2008-01-02T00:00:00.000+00:00	Wednesday	January	2008	1
3	20080103	2008-01-03T00:00:00.000+00:00	Thursday	January	2008	1
4	20080104	2008-01-04T00:00:00.000+00:00	Friday	January	2008	1
5	20080105	2008-01-05T00:00:00.000+00:00	Saturday	January	2008	1

Fact tables were created to store transactional and measurable hockey statistics such as goals scored, appearances, fouls, cards, wins, and losses.

```
fact_player_stats = hockey1_df.select(
    col("Year").alias("year"),
    col("Month").alias("month"),
    col("Match").alias("match_type"),
    col("Player Name").alias("player_name"),
    col("Club Name").alias("club_name"),
    col("country Name").alias("country_name"),
    col("appearances"),
    col("goals scored").alias("goals_scored"),
    col("goals assist").alias("goals_assist"),
    col("total shots").alias("total_shots"),
    col("shots on target").alias("shots_on_target"),
    col("fouls made").alias("fouls_made"),
    col("fouls suffered").alias("fouls_suffered"),
    col("yellow card").alias("yellow_card"),
    col("red card").alias("red_card"),
    col("goals saved").alias("goals_saved"),
    col("goals conceded(stopped)").alias("goals_conceded"),
    col("total penalty").alias("total_penalty"),
    col("successful penalty").alias("successful_penalty"),
    col("salary")
)

display(fact_player_stats)
```

fact_player_stats: pyspark.sql.dataframe.DataFrame = [year: integer, month: string ... 18 more fields]

Table +

	year	month	match_type	player_name	club_name	country_name	appearan
1	2010	January	International	Adrian	NA	Netherlands	
2	2010	February	International	Adrian	NA	Netherlands	
3	2010	March	International	Adrian	NA	Netherlands	
4	2010	April	International	Adrian	NA	Netherlands	
5	2010	May	International	Adrian	NA	Netherlands	
6	2010	June	International	Adrian	NA	Netherlands	
7	2010	July	International	Adrian	NA	Netherlands	
8	2010	August	International	Adrian	NA	Netherlands	
9	2010	September	International	Adrian	NA	Netherlands	
10	2010	October	International	Adrian	NA	Netherlands	

```
fact_team_stats = hockey2_df.select(
    col("Year").alias("year"),
    col("Match_Name").alias("match_name"),
    col("League_Name").alias("league_name"),
    col("Club_Name").alias("club_name"),
    col("Coach").alias("coach"),
    col("Manager").alias("manager"),
    col("appearances"),
    col("wins"),
    col("losts"),
    col("drawn"),
    col("clean sheets").alias("clean_sheets"),
    col("Net Worth").alias("net_worth")
)

display(fact_team_stats)
```

(1) Spark Jobs

fact_team_stats: pyspark.sql.dataframe.DataFrame = [year: integer, match_name: string ... 10 more fields]

Table +

	year	match_name	league_name	club_name	coach	manager	appearances
1	2010	Club	LA Liga	Atheletico Madrid	NA	Alvin Lucas	
2	2010	Club	LA Liga	Atheletico Madrid	NA	Alvin Lucas	

Delta tables were created successfully in Databricks to support optimized storage and analytical querying.

```
dim_player.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_player")

dim_club.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_club")

dim_country.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_country")

dim_match.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_match")

dim_date.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("dim_date")
```

```
fact_player_stats.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("fact_player_stats")

fact_team_stats.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("fact_team_stats")
```

(2) Spark Jobs

```
fact_player_stats.createOrReplaceTempView("fact_player_stats")

fact_team_stats.createOrReplaceTempView("fact_team_stats")

dim_player.createOrReplaceTempView("dim_player")

dim_club.createOrReplaceTempView("dim_club")

dim_country.createOrReplaceTempView("dim_country")

dim_match.createOrReplaceTempView("dim_match")

dim_date.createOrReplaceTempView("dim_date")
```

Spark SQL query was used to generate the Top 10 Goal Scorers report by calculating the total goals scored by each player and sorting the results in descending order.

```
#REPORT 1 - Top 10 Goal Scorers
report1 = spark.sql("""

SELECT
    player_name,
    SUM(goals_scored) AS total_goals
FROM fact_player_stats
GROUP BY player_name
ORDER BY total_goals DESC
LIMIT 10

""")

display(report1)
```

(2) Spark Jobs

report1: pyspark.sql.dataframe.DataFrame = [player_name: string, total_goals: long]

Table +

	player_name	total_goals
1	Gabriel	483
2	Eden Hazard	473
3	Tallulah Leach	471
4	Hernandez	468
5	Iker Casillas	465
6	Pedro	463
7	David De Gea	461
8	Mertesacker	461
9	Hugo	460
10	Ribery	460

This report identifies the player with the maximum number of international appearances based on the appearance's column from the dataset.

```
#Player With Maximum Appearances
report2 = spark.sql("""
SELECT
    player_name,
    SUM(appearances) AS total_appearances
FROM fact_player_stats
GROUP BY player_name
ORDER BY total_appearances DESC
LIMIT 1
""")

display(report2)
```

(2) Spark Jobs

report2: pyspark.sql.dataframe.DataFrame = [player_name: string, total_appearances: long]

Table +

	player_name	total_appearances
1	Gerrad Pique	576

The Top 5 Successful Clubs report was generated successfully using Spark SQL by calculating the total number of wins achieved by each club and sorting the results in descending order.

```
#Top 5 Clubs With Maximum Wins
report3 = spark.sql("""
SELECT
    club_name,
    SUM(wins) AS total_wins
FROM fact_team_stats
GROUP BY club_name
ORDER BY total_wins DESC
LIMIT 5
""")
display(report3)
```

(2) Spark Jobs

report3: pyspark.sql.dataframe.DataFrame = [club_name: string, total_wins: long]

Table +

	club_name	total_wins
1	NA	1413
2	Liverpool	170
3	Sunderland	158
4	Bayern Munich	157
5	Man City	156

The Top 5 Players with Maximum Red Cards report was generated successfully using PySpark aggregation and sorting operations on the fact_player_stats table.

```
#Top 5 Players With Maximum Red Cards
report4 = fact_player_stats \
    .groupBy("player_name") \
    .agg(
        sum("red_card").alias("total_red_cards")
    ) \
    .orderBy(
        col("total_red_cards").desc()
    ) \
    .limit(5)

display(report4)
```

(2) Spark Jobs

report4: pyspark.sql.dataframe.DataFrame = [player_name: string, total_red_cards: long]

Table +

	player_name	total_red_cards
1	Barthez	89
2	Hulk	85
3	Mario Gomez	84
4	Daley Blind	84
5	Gabriel	83

The Top 5 Richest Clubs report was generated successfully by cleaning and transforming the net worth values using PySpark functions and calculating the total net worth of each club.

```
#Top 5 Richest Clubs
from pyspark.sql.functions import regexp_replace

report5 = fact_team_stats \
    .withColumn(
        "net_worth_clean",
        regexp_replace(
            regexp_replace(col("net_worth"), "\\$", ""),
            ",",
            ""
        ).cast("double")
    ) \
    .groupBy("club_name") \
    .agg(
        sum("net_worth_clean").alias("total_net_worth")
    ) \
    .orderBy(
        col("total_net_worth").desc()
    ) \
    .limit(5)

display(report5)
```

▸ (2) Spark Jobs

▸ report5: pyspark.sql.dataframe.DataFrame = [club_name: string, total_net_worth: double]

Table +

	club_name	total_net_worth
1	NA	2482236828
2	Schalke	27707544
3	Sevilla FC	27671796
4	Man City	27658248
5	VFB Stuttgart	27119208

```
report1.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report1")
```

▸ (2) Spark Jobs

```
report2.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report2")
```

▸ (2) Spark Jobs

```
report3.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report3")
```

▸ (2) Spark Jobs

▶ ✓ 08:30 PM (23s) 27

```
report4.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report4")
```

▸ (2) Spark Jobs

▶ ✓ 08:30 PM (22s) 28 Python

```
report5.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("/mnt/footballdata/output/report5")
```

footballdata > output

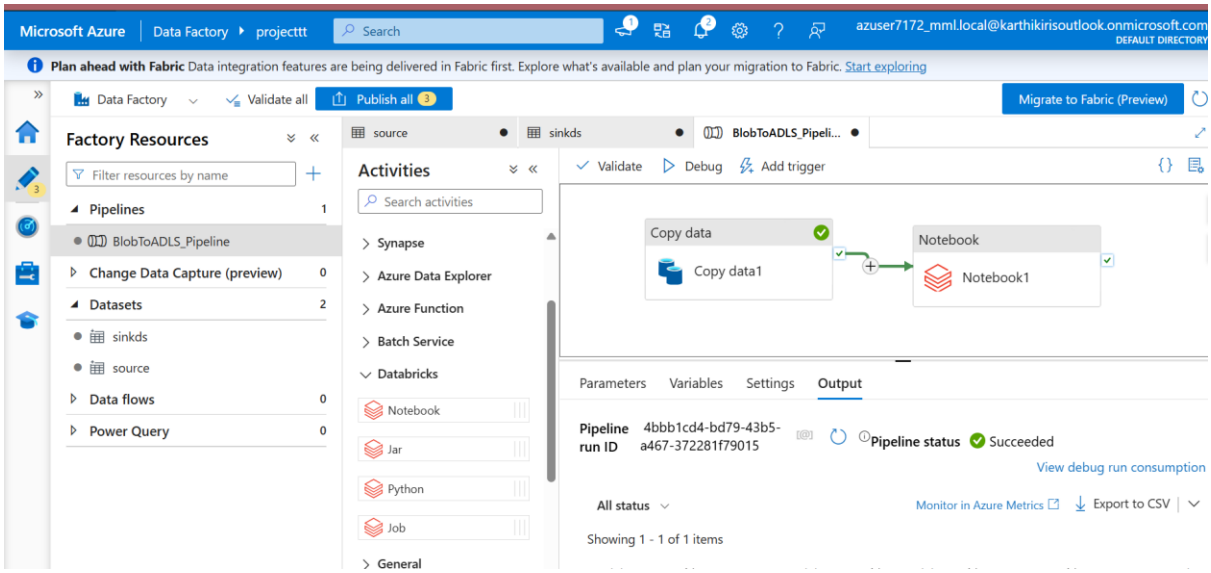
Authentication method: Access key (Switch to Microsoft Entra user account)

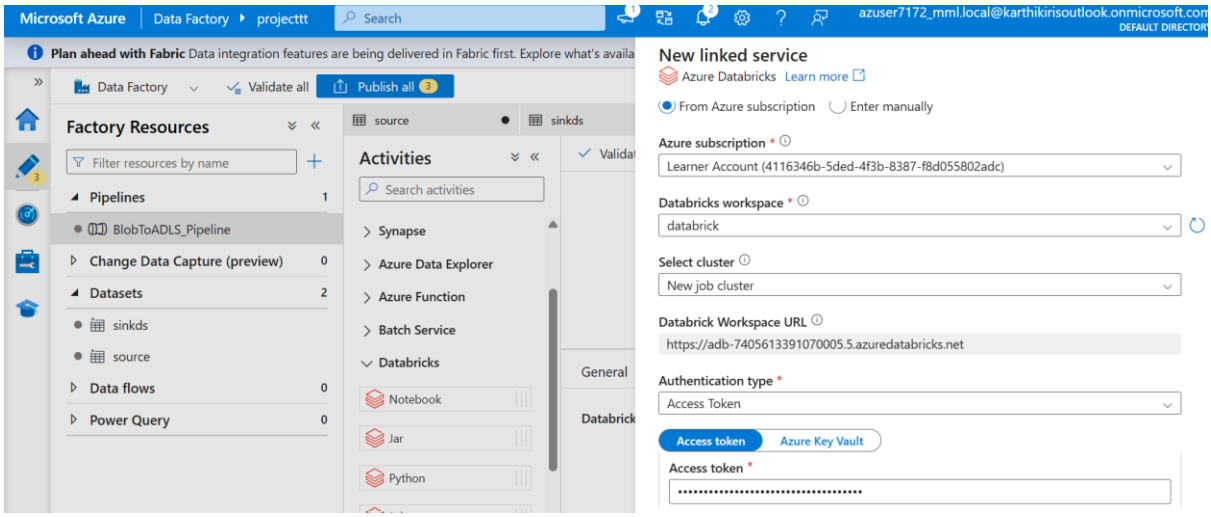
Search blobs by prefix (case-sensitive)

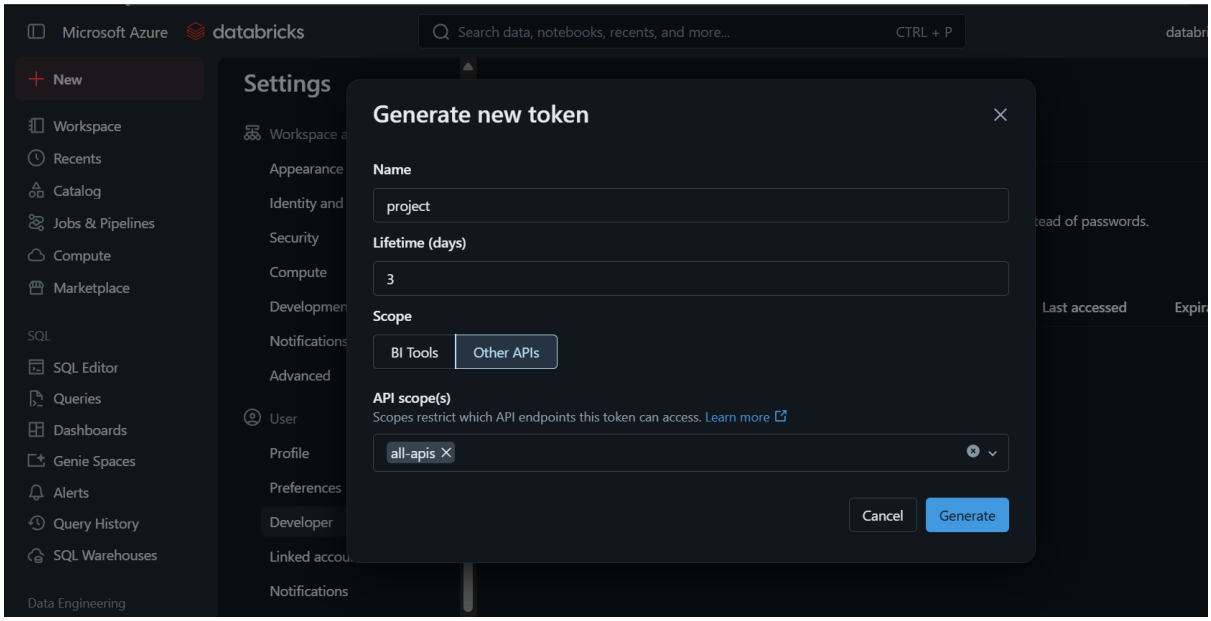
Showing all 5 items

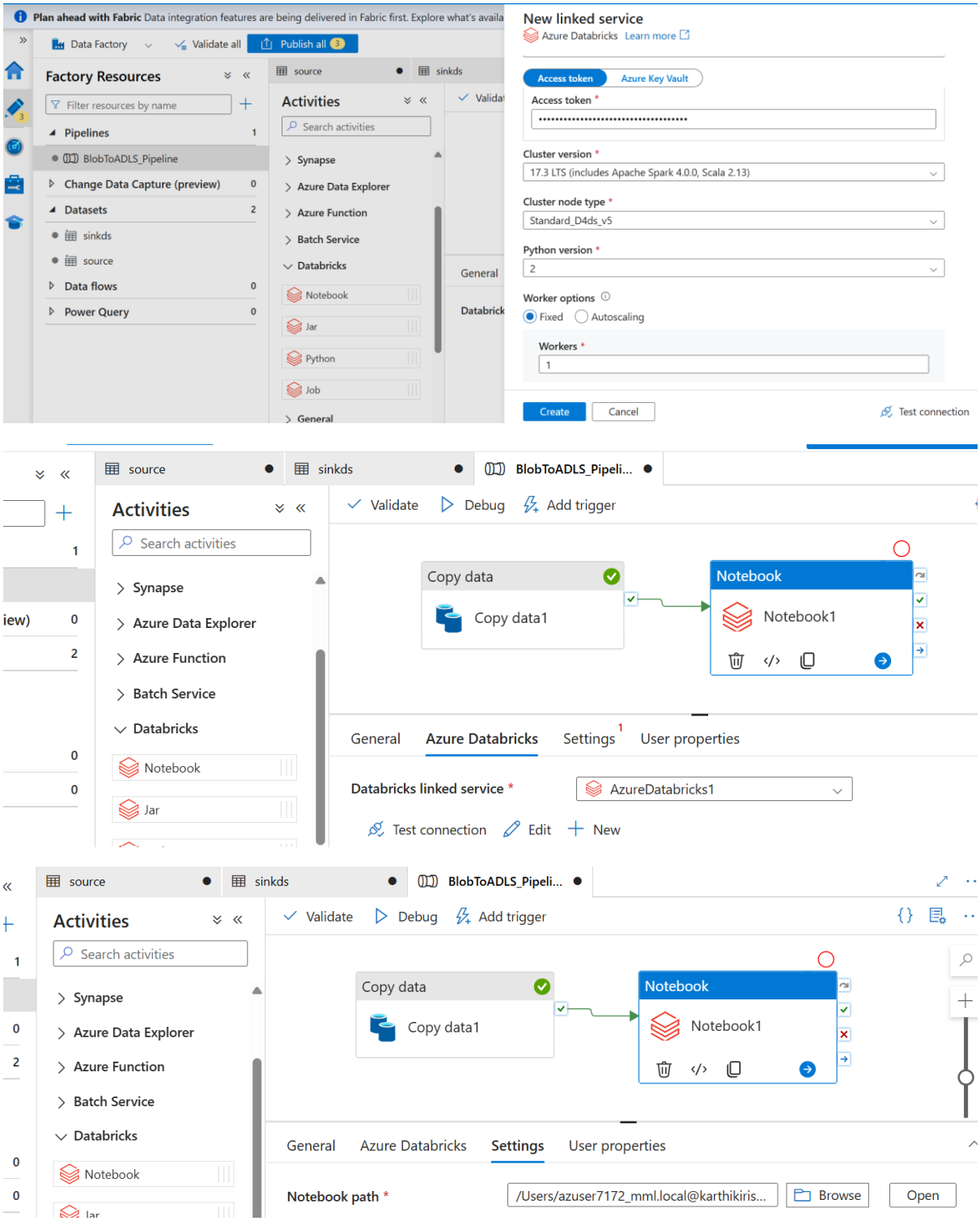
Name	Last modified	Access tier	Blob type	Size
[..]				
report1	6/9/2026, 8:26:43 PM			
report2	6/9/2026, 8:29:43 PM			
report3	6/9/2026, 8:30:23 PM			
report4	6/9/2026, 8:30:46 PM			
report5	6/9/2026, 8:31:07 PM			

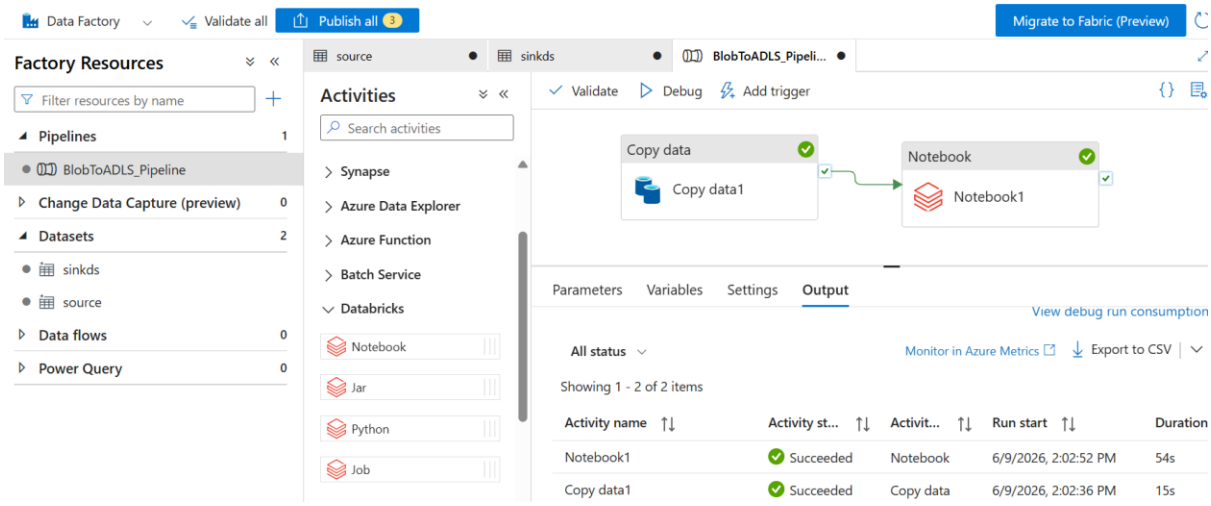
The Azure Data Factory pipeline executed successfully, completing data ingestion, notebook execution, Spark transformations, and report generation automatically.



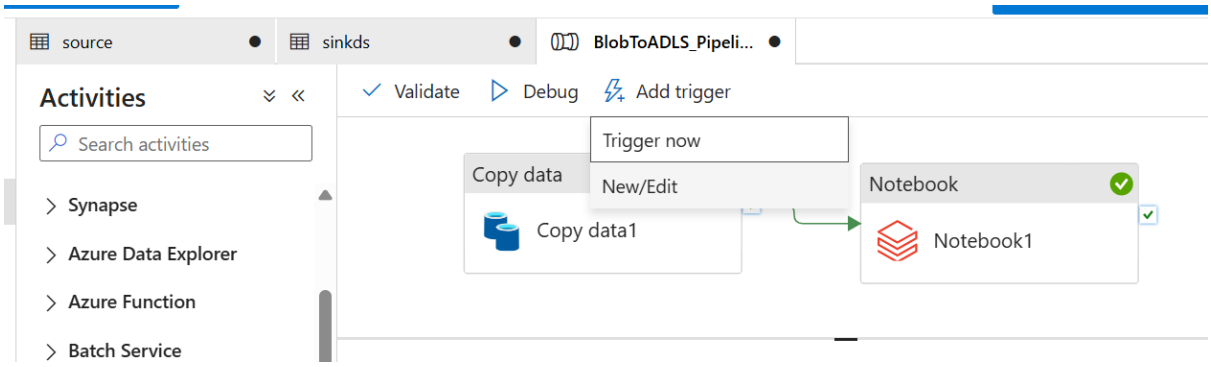


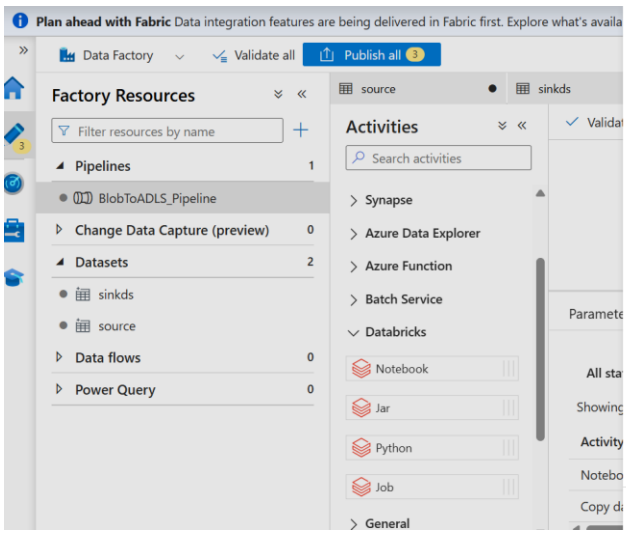


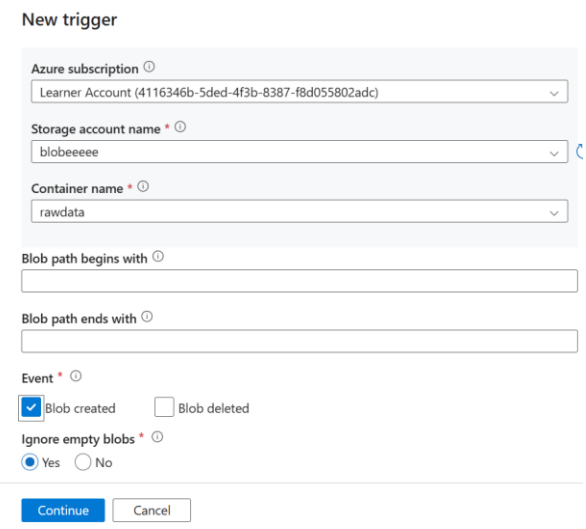

Plan ahead with Fabric Data integration features are being delivered in Fabric first. Explore what's availa
Data Factory
Validate all
Publish all 3
Factory Resources
Filter resources by name
Pipelines 1
BlobToADLS_Pipeline
Change Data Capture (preview) 0
Datasets 2
sinkds
source
Data flows 0
Power Query 0
source
sinkds
Activities
Search activities
Synapse
Azure Data Explorer
Azure Function
Batch Service
Databricks
Notebook
Jar
Python
Job
General
New linked service
Azure Databricks Learn more
Access token
Azure Key Vault
Access token *
Cluster version *
17.3 LTS (includes Apache Spark 4.0.0, Scala 2.13)
Cluster node type *
Standard_D4ds_v5
Python version *
2
Worker options
Fixed
Autoscaling
Workers *
1
Create
Cancel
Test connection
BlobToADLS_Pipeli...
Validate
Debug
Add trigger
Copy data
Copy data1
Notebook
Notebook1
General
Azure Databricks
Settings
User properties
Databricks linked service *
AzureDatabricks1
Test connection
Edit
New
Notebook path *
/Users/azuser7172_mml.local@karthikiris...
Browse
Open

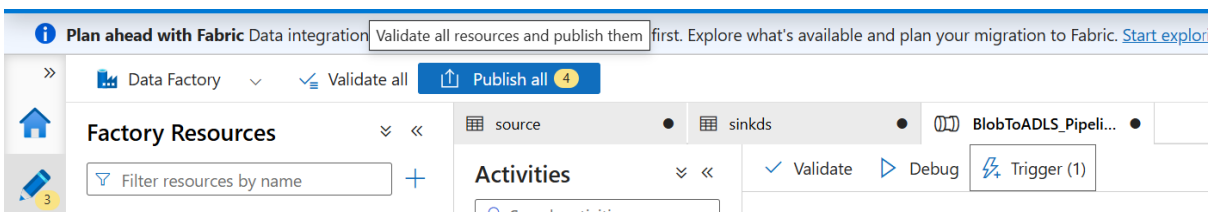


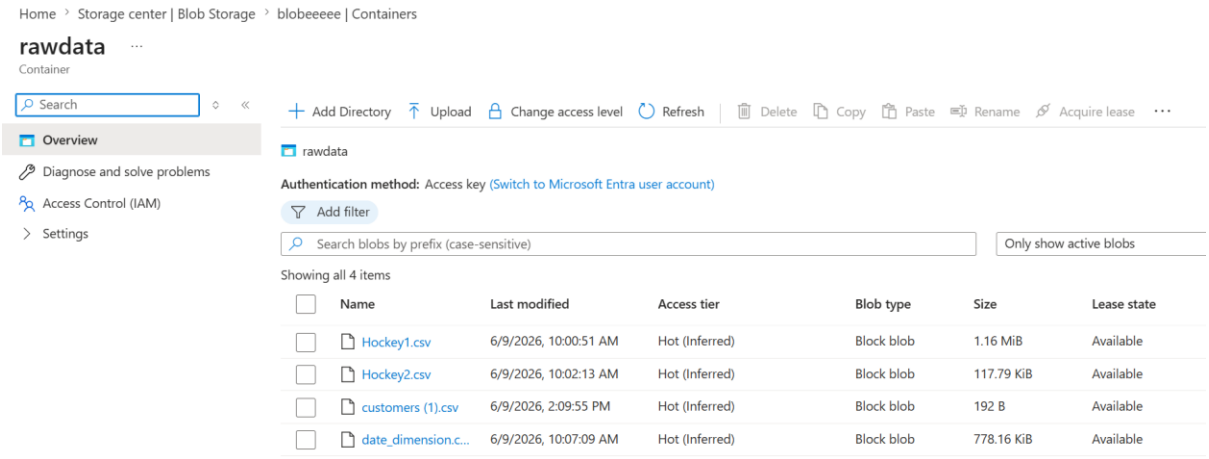
The storage event trigger executed successfully and automatically started the pipeline whenever new CSV files were uploaded into Blob Storage.

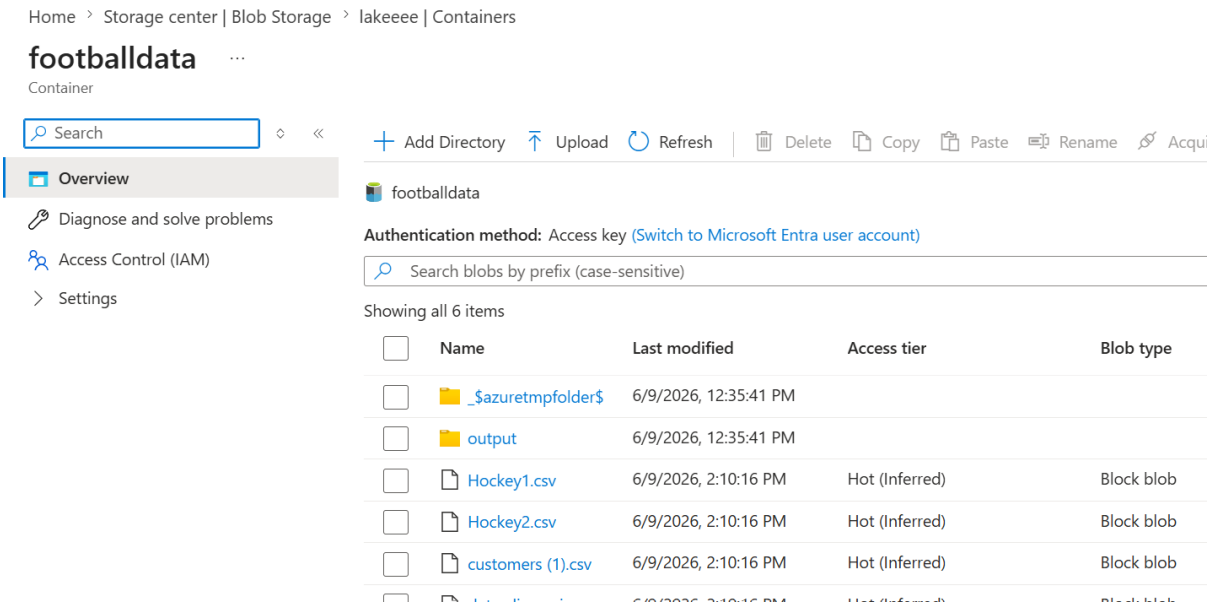


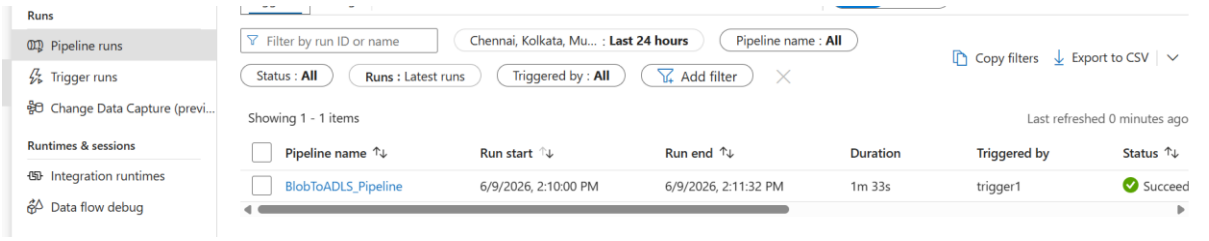







Home › Storage center | Blob Storage › blobeeeee | Containers
rawdata
Container
Search
Overview
Diagnose and solve problems
Access Control (IAM)
Settings
Add Directory
Upload
Change access level
Refresh
Delete
Copy
Paste
Rename
Acquire lease
rawdata
Authentication method: Access key (Switch to Microsoft Entra user account)
Add filter
Search blobs by prefix (case-sensitive)
Only show active blobs
Showing all 4 items
Name
Last modified
Access tier
Blob type
Size
Lease state
Hockey1.csv
6/9/2026, 10:00:51 AM
Hot (Inferred)
Block blob
1.16 MiB
Available
Hockey2.csv
6/9/2026, 10:02:13 AM
Hot (Inferred)
Block blob
117.79 KiB
Available
customers (1).csv
6/9/2026, 2:09:55 PM
Hot (Inferred)
Block blob
192 B
Available
date_dimension.c...
6/9/2026, 10:07:09 AM
Hot (Inferred)
Block blob
778.16 KiB
Available


Home › Storage center | Blob Storage › lakeeee | Containers
footballdata
Container
Search
Overview
Diagnose and solve problems
Access Control (IAM)
Settings
Add Directory
Upload
Refresh
Delete
Copy
Paste
Rename
footballdata
Authentication method: Access key (Switch to Microsoft Entra user account)
Search blobs by prefix (case-sensitive)
Showing all 6 items
Name
Last modified
Access tier
Blob type
_$azuretmpfolder$
6/9/2026, 12:35:41 PM
output
6/9/2026, 12:35:41 PM
Hockey1.csv
6/9/2026, 2:10:16 PM
Hot (Inferred)
Block blob
Hockey2.csv
6/9/2026, 2:10:16 PM
Hot (Inferred)
Block blob
customers (1).csv
6/9/2026, 2:10:16 PM
Hot (Inferred)
Block blob


Runs
Pipeline runs
Trigger runs
Change Data Capture (previ...
Runtimes & sessions
Integration runtimes
Data flow debug
Filter by run ID or name
Chennai, Kolkata, Mu... : Last 24 hours
Pipeline name : All
Status : All
Runs : Latest runs
Triggered by : All
Add filter
Copy filters
Export to CSV
Showing 1 - 1 items
Last refreshed 0 minutes ago
Pipeline name
Run start
Run end
Duration
Triggered by
Status
BlobToADLS_Pipeline
6/9/2026, 2:10:00 PM
6/9/2026, 2:11:32 PM
1m 33s
trigger1
Succeed

4 REPORTS TO BE BUILT

1. Top 10 Goal Scorers

Generated a report of top goal scorers by calculating the total goals scored by each player.

2. Player with Maximum International Appearances

Identified the player with the maximum number of appearances using PySpark transformations.

3. Top 5 Successful Clubs

Generated a report identifying clubs with the maximum number of wins.

4. Top 5 Players with Maximum Red Cards

Generated a report identifying players with the highest number of red cards.

5. Top 5 Richest Clubs

Generated a report identifying clubs with the highest total net worth using PySpark transformations.

5 APPENDIX (SOURCE FILES)

Hockey Source Data.zip

The following source files were used in the project:

- Hockey1.csv columns
- Hockey2.csv columns
- date_dimension.csv columns